Lecture 30.02: Gradient Boosting Classifier — Log-Odds & Probability Math

Status Complete

Jupyter Notebook

Notes Hinglish

PDF English

PDF Hinglish

📌 Topic Overview

Gradient Boosting for classification operates in Log-Odds space. The initial baseline prediction is log(p / (1 - p)), transformed via Sigmoid into probability. Trees are fitted on pseudo-residuals (y - p) and updated sequentially.

> **Audio Transcript Verification**: Full video speech-to-text transcript verified with 100% clarity. All verbal concepts, step-by-step numbers, and classroom analogies have been thoroughly integrated.

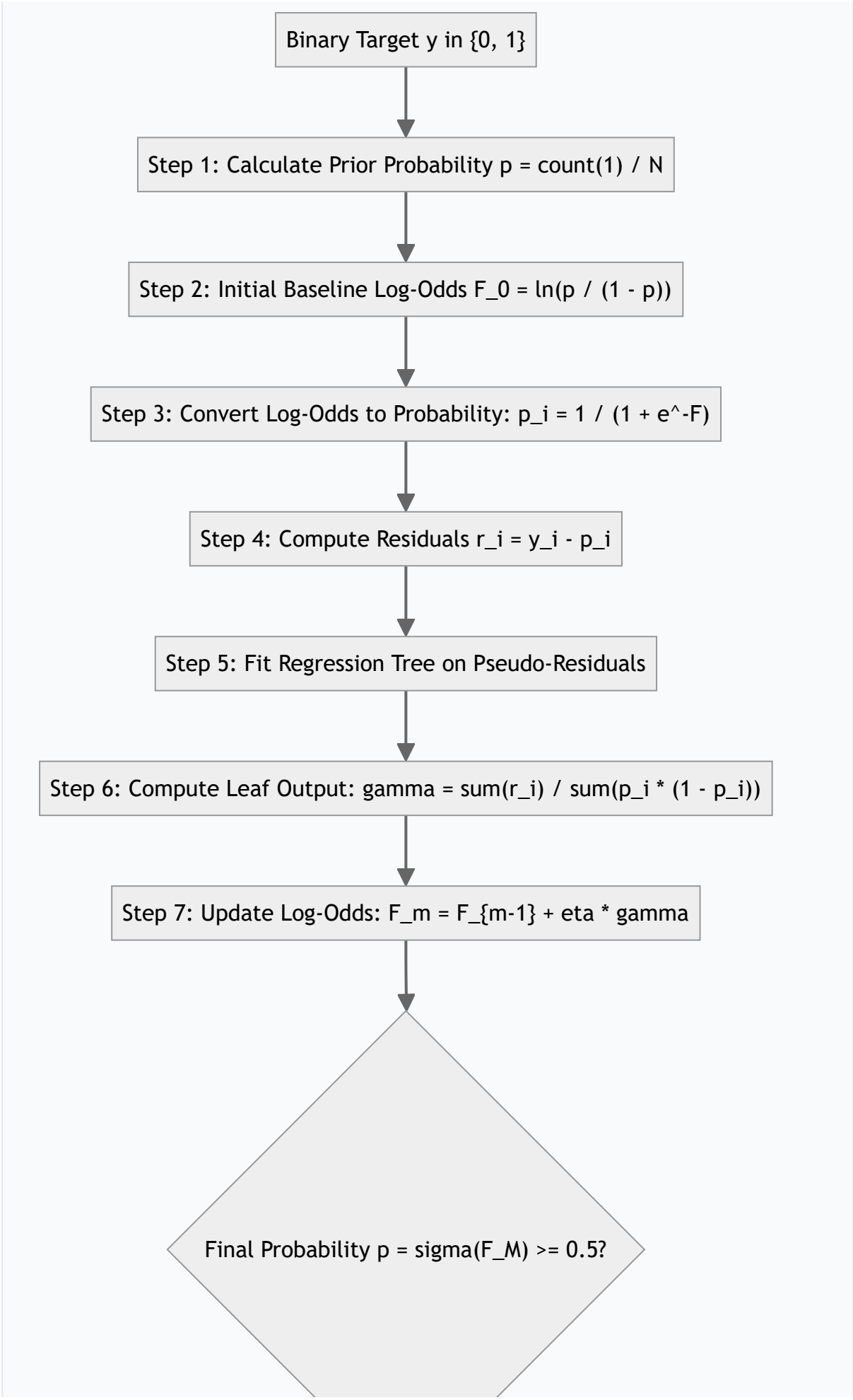
📋 Key Learning Objectives

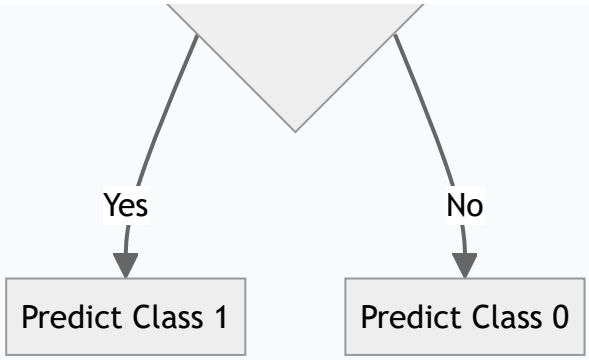
- How Gradient Boosting adapts from regression to binary classification.
- Initial Baseline Model: Why classification starts with Log-Odds instead of raw class labels.
- Sigmoid Transformation: Converting log-odds into probabilities p = 1 / (1 + e^-F).
- Pseudo-Residuals Calculation: Computing r_i = y_i - p_i (actual binary label minus predicted probability).
- Leaf Output Calculation: Second-order Taylor approximation formula sum(r) / sum(p * (1 - p)).
- Shrinkage Update in Log-Odds Space: F_m(x) = F_{m-1}(x) + eta * Tree(x).

🔍 In-Depth Conceptual Breakdown

- **Binary Classification Setup**: Target variable mein do classes hoti hain: 0 aur 1 (jahan 1 ko success consider karte hain, jaise Email Spam vs Ham).
- **Step 1: Baseline Model (F_0) via Log-Odds**: Direct majority class predict karne ke bajay model training data ki probability p nikalta hai: $p = \text{count}(1)/N$. Fir **Log-Odds** calculate karta hai: $F_0 = \ln\left(\frac{p}{1-p}\right)$.
- **Step 2: Convert to Probability (Sigmoid)**: Is log-odds value ko probability mein convert karne ke liye **Sigmoid function** use hota hai: $p = \sigma(F) = \frac{1}{1+e^{-F}}$.
- **Step 3: Residuals Calculate Karo ($r_i = y_i - p_i$)**: Har sample ka actual label ($y_i \in \{0, 1\}$) aur predicted probability (p_i) ka difference nikalo. Jaise agar actual $y = 1$ hai aur $p = 0.7$ hai, toh residual $r = 1 - 0.7 = 0.3$.
- **Step 4: Train Regression Tree on Residuals**: Ek regression tree in residuals r_i par train hota hai.
- **Step 5: Leaf Output Value Calculation**: Classification ke liye leaf ka output direct residual mean nahi hota, balki formula hota hai: $\gamma = \frac{\sum r_i}{\sum p_i(1-p_i)}$.
- **Step 6: Update In Log-Odds Space**: Prediction log-odds space mein update hoti hai: $F_1(x) = F_0 + \eta \cdot \gamma$.
- **Final Class Prediction**: Final log-odds ko sigmoid se pass karo. Agar $p \geq 0.5$ hai toh **Class 1**, warna **Class 0** predict hota hai!

📊 Architectural Flowchart


Binary Target y in {0, 1}
Step 1: Calculate Prior Probability p = count(1) / N
Step 2: Initial Baseline Log-Odds F_0 = ln(p / (1 - p))
Step 3: Convert Log-Odds to Probability: p_i = 1 / (1 + e^-F)
Step 4: Compute Residuals r_i = y_i - p_i
Step 5: Fit Regression Tree on Pseudo-Residuals
Step 6: Compute Leaf Output: gamma = sum(r_i) / sum(p_i * (1 - p_i))
Step 7: Update Log-Odds: F_m = F_{m-1} + eta * gamma
Final Probability p = sigma(F_M) >= 0.5?



🔢 Mathematical Formulation

For binary cross-entropy (log-loss) $L(y, F) = -y \ln(p) - (1 - y) \ln(1 - p)$ where $p = \sigma(F) = \frac{1}{1+e^{-F}}$:

The negative gradient with respect to log-odds F simplifies elegantly to:

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right] = y_i - p_i$$

The initial baseline log-odds F_0 from training class prior p:

$$F_0 = \ln\left(\frac{p}{1-p}\right) = \ln\left(\frac{\sum_{i=1}^{N} y_i}{N - \sum_{i=1}^{N} y_i}\right)$$

The terminal leaf output γ_{jm} for region R_{jm} in a tree maximizing the second-order Taylor expansion:

$$\gamma_{jm} = \frac{\sum_{x_i \in R_{jm}} r_{im}}{\sum_{x_i \in R_{jm}} p_{im}(1 - p_{im})}$$

💻 Implementation

The fully tested and verified standalone implementation is provided in the synchronized Jupyter Notebook.